

T.U.H.L

To Unravel Historic Literature

Qualitätsicherung

Jan Hirschmann Nicoletta Pütz Kathrin Leonie Schmidt
Maximilian David Walz

Sommersemester 2020, 30. August

Inhaltsverzeichnis

1	Testbericht	3
1.1	Funktionsfehler	3
1.2	Performanceverbesserungen	8
1.3	Verbesserung der Nutzererfahrung	9
1.4	Fehler deren Implementierung später nochmal verändert wurde	10
2	Neue Funktionen	12

1 Testbericht

Im Folgenden werden alle in der Qualitätsicherung gefundenen und behobenen Fehler aufgeführt. Hierzu wird zunächst das Symptom beschrieben, dann der Grund für den Fehler und anschließend die unternommenen Maßnahmen zur Fehlerbehebung. Diese sind gegliedert in Funktionsfehler, Performanceverbesserungen und Bequemlichkeitsverbesserungen.

1.1 Funktionsfehler

BF 01: Ändern der Sprache

Symptom: Das Ändern der Sprache funktioniert nicht.

Grund: Die Datei, welche die deutsche Version der Texte enthält, folgt nicht den Benennungsschema.

Behebung: Umbenennung der Datei, sodass die Benennung dem Schema folgt: Von „messages_DE.properties“ in „messages_de.properties“.

BF 02: Zurücksetzen des Suchbegriffs

Symptom: Leeren des Eingabefeldes für die Suche hat die Ergebnisse nicht geändert.

Grund: Bei einer leeren Eingabe wurde die GET-Anfrage zum Ändern des aktuellen Suchbegriffe nicht ausgeführt.

Behebung: Entfernen der entsprechenden if-Bedingung, die verhinderte, dass eine leere Eingabe zu einer GET-Anfrage führt.

BF 03: Reihenfolge beim Anwenden von Filtern

Symptom: Gefilterte Ergebnisse sind abhängig von der Reihenfolge in der die Filter angewendet wurden.

Grund: Die Elasticsearchanfragen wurden aneinandergehängt. Damit war es teilweise möglich, dass ein Suchergebnis nur eine der Filterbedingungen erfüllt.

Behebung: Verwendung der `bool query`. Diese erlaubt es zu spezifizieren, dass alle Anfragen welche Filter repräsentieren mit einem logischen UND verknüpft sind. So müssen alle Filterbedingungen zutreffen, damit ein Ergebnis weiterhin angezeigt wird.

BF 04: Formatierung der Tabelle beim Ausblenden von Spalten

Symptom: Die Formatierung der Tabelle geht kaputt wenn alle ausblendbaren Spalten ausgeblendet sind.

Grund: Dokumentation Tabulator: Note: At least one column must not be frozen for the table to render correctly. If you want to freeze all your columns then you will achieve the same effect by not freezing any. Die erste Spalte ist „frozen“, damit sie nicht bewegt werden kann.

Behebung: Da eine neue Spalte mit Schaltflächen hinzugefügt wurde ist jetzt nur noch diese „frozen“ und die Id nicht mehr. Daher ist die Spalte mit der ID jetzt verschiebbar aber immernoch nicht ausblendbar. So ist niemals nur die „frozen“ Spalte in der Tabelle und die Formatierung geht nicht mehr kaputt. (Es wurde erst probiert beide Spalten, also Schaltflächen und ID, „frozen“ zu machen und im Fall, dass alle anderen Spalten versteckt sind, diese zu „unfreeze“. Allerdings hat das „unfreeze“ auch die Formatierung der Tabelle kaputt gemacht.)

BF 05: Zurücksetzen der Spalten hat nicht das Dropdownmenü aktualisiert

Symptom: Beim Wiedereinblenden aller Spalten mithilfe der Spalten zurücksetzen Schaltfläche wurden die Checkboxes im Dropdownmenü zum Auswählen der Spalten nicht aktualisiert.

Grund: In diesem Fall wurde vergessen das Dropdownmenü zu updaten.

Behebung: Eine Methode diese upzudaten wurde implementiert und wird nun in diesem Fall aufgerufen.

BF 06: Persistenz von einzelnen Vorschaubildern

Symptom: Wenn man einzeln Vorschaubilder ein- oder ausblendet und dann die Seite neu lädt, bleiben diese nicht persistent.

Grund: Es wurde bis jetzt nur gespeichert, wenn alle Vorschaubilder ausgeklappt waren.

Behebung: Im User wird jetzt zusätzlich eine Liste mit Strings abgespeichert, die die Reihen beinhaltet, deren Vorschaubilder gerade angezeigt werden.

BF 07: Filter leeren lädt Tabelle nicht neu

Symptom: Beim Betätigen der „Filter leeren“ Schaltfläche wird die Tabelle nicht neu geladen, sondern man muss erst nochmal „Filter anwenden“ betätigen.

Grund: Implementierung entsprach den geplanten Funktionen im Pflichtenheft.

Behebung: Intern wird die Methode `applyFilters()` zusätzlich aufgerufen.

BF 08: addAnnotation im AccessService

Symptom: `addAnnotation` wirft eine `NullPointerException`.

Grund: Es wird in `buildJsonFromAnnotation()` immer `getAnnotationById()` aufgerufen, aber in `addAnnotation()` erzeugt die ID erst später.

Behebung: `getAnnotationById()` wird nur unter der Bedingung aufgerufen, dass es eine ID gibt.

BF 09: Anwenden der Suche setzt die aktuelle Seite der Tabelle nicht zurück

Symptom: Beim Anwenden der Suche wird die aktuelle Seite der Tabelle nicht auf die erste Seite zurückgesetzt.

Grund: Die `currentPage` im `TableViewService` wird nicht überschrieben.

Behebung: Mithilfe einer neuen HTTP Anfrage die `currentPage` im `TableViewService` wird diese neu gesetzt.

BF 10: Exceptions auf der Errorpage anzeigen (JSON)

Symptom: In einigen Fällen wurden Exceptions beim parsen von JSON in der Konsole aufgerufen und haben im Frontend nicht die Errorpage aufgerufen.

Grund: Wurde übersehen.

Behebung: Mithilfe eines Workarounds wird die Errorpage nun im success-callback der HTTP Anfrage bei Bedarf aufgerufen.

BF 11: Änderung der auswählbaren Seitengröße

Symptom: Bei der Auswahl der angezeigten Ergebnisse pro Seite kommen unterschiedliche Auswahlmöglichkeiten. Anfangs kann man 10, 20, 30 und 40 wählen, wenn man die Seite neu lädt und eine höhere Zahl gewählt hatte wird diese zum Minimum.

Grund: Die Auswahl wird vom Tabulator dynamisch generiert.

Behebung: Feste Vorgabe der Auswahl in der Definition der Tabelle.

BF 12: Speichern der Creator für Annotationen

Symptom: Creator werden teilweise doppelt, teilweise gar nicht, teilweise sehr kryptisch abgespeichert.

Grund: Fallunterscheidung in `buildCreators` war falsch und einige Fallunterscheidungen haben gefehlt. **Behebung:** Fallunterscheidung wurde überarbeitet, in weitere Methoden ausgelagert und kommentiert um sie übersichtlicher und verständlicher zu machen.

BF 13: Target in Annotation wird nicht abgespeichert

Symptom: SVGCode für neue Annotation wird nicht in Annotationsstore gespeichert.

Grund: JSON Objekte, wie das „Target“ Objekt, werden nur erstellt, wenn das vorher bereits abgespeicherte JSON Objekt zu der Annotation sie bereits hatte. Bei neuen Annotationen sind sie daher leer.

Behebung: JSONObjekte werden unabhängig davon erstellt.

BF 14: Speichern der Annotationen im Annotationsstore

Symptom: Annotationen werden nicht im Annotationsstore gespeichert, aber im Suchindex.

Grund: HTTP POST Request wurde nicht vom Annotationsstore angenommen.

Behebung: Ein Header mit `content type: application/json` wurde hinzugefügt.

BF 15: Annotationen aus beiden Containern im Index

Symptom: Annotationen, die sowohl im „validated“ Container als auch „deInterpretation“ enthalten sind kommen doppelt im Index vor.

Grund: Die Methoden, die Annotationen über SPARQL Queries holen kontrollieren nicht, ob die Annotationen doppelt enthalten sind.

Behebung: Die Canonical IDs werden ausgelesen und mit den geholten Annotationen verglichen, danach werden die redundanten Annotationen gelöscht.

BF 16: Annotationen können nicht im Annotationsstore gespeichert werden

Symptom: Annotationen werden nach dem updaten nicht wieder im Annotationsstore gespeichert.

Grund: Der Etag von den geupdateten Annotationen wurde nicht/falsch gespeichert. Daher wurden sie beim erneuten Speichern nicht vom Annotationsstore abgelehnt.

Behebung: Der Etag wird dem Annotations-JSON direkt nach dem Abrufen hinzugefügt und vor dem erneuten Speichern wieder aus dem JSON entfernt.

BF 17: Farben von Annotationen

Symptom: Farben von Annotationen werden nicht im Annotationsstore abgespeichert, sind aber im Suchindex zu sehen.

Grund: Die Farbe wird nicht im JSON abgespeichert, das in den Annotationsstore geladen wird.

Behebung: Die Farbe wird jetzt dem JSON genau einmal hinzugefügt, oder es überschreibt die bereits vorhandene Farbe.

BF 18: Annotation mit Motivation nicht im Annotationsstore gespeichert

Symptom: Eine Annotation wird nicht im Annotationsstore gespeichert.

Grund: Die Motivation der Annotation entspricht nicht einer durch das WADM festgelegten Liste an Wörtern.

Behebung: Ein Enum mit diesen Wörtern wurde dem Model hinzugefügt und nun kann der User nur diese Motivationen hinzufügen.

BF 19: Pufferüberlauf in der Antwort einer Suchanfrage

Symptom: Eine Exception wird geworfen, wenn manche Tabellenseiten aufgerufen werden.

Grund: Wenn die auf einer Tabellenseite aufgeführten Manuskripte besonders viele Seiten und Annotationen haben ist die Antwort, die von Elasticsearch auf die Suchanfrage kommt, zu groß für die Antwortpuffergröße die Spring Data Elasticsearch benutzt.

Behebung: Der `RestHighLevelClient` wird verwendet um die Suchanfrage an den Elasticsearch Server auszuführen. Auf diese Weise kann man die Suchanfrage mehr spezifizieren. So kann zum Beispiel angegeben werden, dass die Annotationen in der Antwort gar nicht mitgeliefert werden. Diese werden nämlich zum Darstellen in der Tabelle nicht benötigt. Damit ist die Antwort um ein vielfaches kleiner, was auch die Geschwindigkeit beim Suchen verbessert.

BF 20: Verhalten der Suche bei Prefixen

Symptom: Wenn nach einem Prefix oder Suffix eines existierenden Titels gesucht wird, werden Manuskripte mit diesem Titel nicht angezeigt.

Grund: Damit Elasticsearch in einem solchen Fall auch Manuskripte anzeigt zu denen der Suchbegriff ein Prefix ihres Titels ist, werden Wildcard Operatoren verwendet. **Behebung:** Dem Suchbegriff wird vorne und hinten der Wildcard Operator `*` hinzugefügt. Dies sorgt dafür, dass beliebig viele Zeichen davor oder dahinter vom Suchbegriff abweichen dürfen.

1.2 Performanceverbesserungen**BF 21: Vorschaubilder nur laden wenn nötig**

Symptom: Die Vorschaubilder werden immer komplett geladen wenn die Tabelle neu geladen wird und teilweise ausgeklappt.

Grund: Implementierung der Vorschaubilder in einer Tabulator `Subtable`.

Behebung: Änderung der Implementierung der Vorschaubilder in einem normalem `div`, mit eigenem HTML.

BF 22: Methode `search()` wird sehr oft aufgerufen

Symptom: Die Methode `search()` wird beim Laden der Tabelle bis zu 16 Mal aufgerufen.

Grund: Die Methode wird aufgerufen um die Ergebnisse zu laden, obwohl sie dafür nicht unbedingt gebraucht wird.

Behebung: Methode wird nur noch aufgerufen wenn wirklich nötig.

BF 23: Veränderung der SPARQL Anfrage

Symptom: SPARQL Anfragen an den Annotationsstore sind sehr langsam.

Grund: Die Anfrage entfernt über das Schlagwort `MINUS` gelöschte Annotationen.

Behebung: Das Ersetzen von `MINUS` durch `FILTER NOT EXISTS` verkürzt die Antwortzeit des Annotationsstores deutlich.

BF 24: Doppelte Annotationen beim Initialisieren entfernen

Symptom: Das Abrufen der Annotation aus dem Container „deInterpretatione“ dauert pro Annotation deutlich länger als das Abrufen der Annotationen aus dem Container „validated“.

Grund: In der Methode `getAllAnnotations()` im `AnnotationStoreAccessService` wird für jede Annotation aus dem Container „deInterpretatione“ alle Annotationen aus dem Container „validated“ durchgegangen, die IDs aus dem „canonical“-Feld verglichen und nur die, die noch nicht enthalten sind hinzugefügt.

Behebung: Die IDs, die in den „canonical“-Feldern der validated-Annotationen enthalten sind werden vorher in eine separate Liste gespeichert und für jede neue Annotation aus `deInterpretatione` über den `contains()` Befehl der Liste abgeglichen.

BF 25: Manuskripte initialisieren

Symptom: Manuskripte brauchen sehr lange beim initialisieren.

Grund: Alle Elemente des Repositoriums werden beim Laden der Manuskripte abgerufen, aber nur die Manuskripte werden gespeichert.

Behebung: Änderung der Anfrage an das Repositorium, sodass nur Manuskripte und keine anderen JSON-Dateien abgerufen werden.

1.3 Verbesserung der Nutzererfahrung**BF 26: Suchfeld beim Hinzufügen neuer Filter**

Symptom: Das Suchfeld beim Hinzufügen neuer Filter ist missverständlich.

Grund: Man hält es intuitiv für ein Eingabefeld für einen Filterwert.

Behebung: Entfernen des Suchfeldes, da die Anzahl an Filtern sowieso überschaubar ist.

BF 27: Bei keinen Suchergebnisse wird keine Nachricht angezeigt

Symptom: Bei keinen Suchergebnissen in der Tabelle ist diese einfach nur leer.

Grund: Noch nicht implementiert.

Behebung: `placeholder` Attribut in Tabelle hinzufügen um eine Nachricht anzuzeigen falls keine Ergebnisse vorhanden sind.

BF 28: Zu viele Klicks in Tabelle

Symptom: In der Tabelle gibt es zu viele Klicks. Doppelklick auf ID, Doppelklick auf Zeile, Klick auf Vorschaubild.

Grund: Implementierung entsprach den geplanten Funktionen im Pflichtenheft.

Behebung: Schaltflächen am Anfang der Zeile und Schaltflächen bei den Vorschaubildern zum Öffnen des Editors.

BF 29: Je nach Eingabe schlägt Hinzufügen der Annotation fehl

Symptom: Wenn im **Motivation**-Feld beim Hinzufügen einer Annotation oder dem **Purpose**-Feld beim Hinzufügen einer Textkarte ein beliebiger Wert eingegeben wird schlägt die Anfrage fehl.

Grund: Der Annotationsstore akzeptiert nur bestimmte Schlagworte als Eingabe für diese Felder.

Behebung: Über ein neu hinzugefügtes Enum werden dem User nur gewisse Schlagworte zur Verfügung gestellt, aus denen er Auswählen kann.

1.4 Fehler deren Implementierung später nochmal verändert wurde**BF 30: Filterkonfiguration nicht mehr speichern**

Symptom: Wenn man das Häkchen nicht mehr gesetzt hatte wurde die Konfiguration trotzdem noch gespeichert.

Grund: Filter im **User** wurden nicht zurückgesetzt.

Behebung: Filter im **User** zurücksetzen.

BF 31: Pagination Index 0 bei keinen Ergebnissen

Symptom: Bei einer Suchanfrage welche keine Ergebnisse liefert wird bei der Pagination als letzte Seite 0 angezeigt und wirft beim darauf klicken eine Exception.

Grund: Es wird immer die Anzahl der Seiten übergeben und angezeigt ohne zu überprüfen ob diese 0 sind. Mit dieser 0 wird dann eine Anfrage gestartet welche schief geht da die Anfrage keine Zahl kleiner 1 enthalten darf.

Behebung: Überprüfen ob die Anzahl der Seiten 0 ist und übergebe in diesem Fall eine 1.

BF 32: Die Schaltflächen nächste und vorherige können die maximale Anzahl an Seiten überschreiten/unterschreiten

Symptom: Mit der nächste Seite Schaltfläche kann man den Index der maximalen Seiten überschreiten, mit der vorherige Seite Schaltfläche unterschreiten, was eine Exception auslöst.

Grund: Wurde noch nicht überprüft.

Behebung: Überprüfen der Werte und deaktivieren der Schaltflächen auf der jeweils letzten und ersten Seite.

2 Neue Funktionen

Während der Qualitätsicherungsphase wurden außerdem aus verschiedenen Gründen neue Funktionen implementiert. Diese werden im folgenden Abschnitt aufgeführt. Es wird jeweils der Grund und die Funktion näher erläutert.

NF 01: Buttons zum Öffnen des Editors

Grund: Die vielen verschiedenen Klicks in der Tabelle waren zu unübersichtlich und verwirrend.

Funktion: In der Tabelle gibt es nun eine Spalte mit Schaltflächen zum Öffnen der ersten Seite im Editor. Außerdem gibt es bei jedem Vorschaubild eine Schaltfläche zum Öffnen des Editors auf der entsprechenden Seite. Über einen Doppelklick auf eine Reihe werden die Vorschaubilder aus-/eingeklappt.

NF 02: Allgemeine Persistenz

Grund: Die Persistenz der Tabelle, Filter und Suche war sehr kompliziert von Grund auf implementiert.

Funktion: Die Persistenz ist jetzt über die vorhandenen Funktionen von Tabulator implementiert, auch die Pagination läuft jetzt automatisch über Tabulator.

NF 03: Format der Spalte hat Algorithmusannotationen

Grund: Die Spalte enthält `true` und `false` Werte, welche durch Icons besser verständlich sind.

Funktion: Die Ergebnisse werden jetzt in Form von Haken und Kreuzen angezeigt.

NF 04: Einstellen der Anzahl an Ergebnissen pro Seite

Grund: Praktische Zusatzfunktion.

Funktion: Man kann die Anzahl der angezeigten Ergebnisse auf einer Seite in einem kleinen Dropdownmenü am unteren Rand der Tabelle auswählen. Zur Auswahl stehen 10, 20, 30 und 40.

NF 05: Editor Stub

Grund: Um das Editor Backend einfach zugänglich zu machen und das Testen seiner Funktionen einfacher zu ermöglichen.

Funktion: Der Editor Stub ist über den Pfad `/editor_stub` zugänglich.

Für Annotationen, Textkarten und Tags werden die CRUD Funktionalitäten angeboten. Hierfür lässt sich eine Maske ausklappen, die aus den notwendigen Feldern besteht. Man kann die Felder dann ausfüllen und die Anfrage abschicken. Auf der rechten Seite sieht man die Rückgabe der Anfrage, also beispielsweise die aktualisierte Annotation beim Abschicken einer Aktualisierungsanfrage.

Zudem gibt es noch Masken für das validieren von Annotationen und das Lesen von rohen Manuskript-, Annotation- und Seitendaten aus dem Repositorium beziehungsweise dem Annotationsstore.